

DEVELOPERS HUB CORPORATION

Cybersecurity Internship Program

WEEKS 4–6 REPORT

Advanced Threat Detection | Ethical Hacking | Security Audits & Deployment

Prepared By: Ayesha Sarwar Khan

Organization: Developers Hub Corporation

Internship: Cybersecurity Internship

Submission Date: July 12, 2026

Duration: July 10 – July 12, 2026

Application: vulnerable-node (Node.js/Express)

CONFIDENTIAL — FOR INTERNSHIP ASSESSMENT PURPOSES ONLY

TABLE OF CONTENTS

WEEK 4 — Advanced Threat Detection & Web Security Enhancements

- 4.1 Executive Summary
- 4.2 Scope & Methodology
- 4.3 Tools & Libraries Used
- 4.4 Task 1 — Intrusion Detection & Monitoring (Fail2Ban)
- 4.5 Task 2 — API Security Hardening
- 4.6 Task 3 — Security Headers & CSP Implementation
- 4.7 Files Modified & Packages Installed
- 4.8 Conclusion

WEEK 5 — Ethical Hacking & Exploiting Vulnerabilities

- 5.1 Executive Summary
- 5.2 Environment Setup
- 5.3 Reconnaissance
- 5.4 SQL Injection Testing
- 5.5 CSRF Protection Implementation
- 5.6 Bugs Discovered During Testing
- 5.7 Files Modified
- 5.8 Conclusion

WEEK 6 — Advanced Security Audits & Final Deployment Security (Partial)

- 6.1 Executive Summary
- 6.2 Task 1 — Security Audits & Compliance (Complete)
- 6.3 Task 2 — Secure Deployment Practices (Partially Complete)
- 6.4 Task 3 — Final Penetration Testing (Not Started)
- 6.5 Overall Week 6 Status Summary
- 6.6 Conclusion

Overall Conclusion

WEEK 4 — ADVANCED THREAT DETECTION & WEB SECURITY ENHANCEMENTS

Application: vulnerable-node (Node.js/Express) | Tools: Fail2Ban, express-rate-limit, cors, Helmet.js | Date: July 10–11, 2026

Attribute	Detail
Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	July 10–11, 2026
Application Tested	vulnerable-node (Node.js/Express)
Assessment Type	Threat Detection Setup & API/Web Security Hardening
Tools Used	Fail2Ban, express-rate-limit, cors, Helmet.js, curl

4.1 Executive Summary

This report documents the security work completed during Week 4 of the Cybersecurity Internship at Developers Hub Corporation. Building on the Week 1–3 foundation, Week 4 focused on three areas: real-time intrusion detection at the host level, hardening the application's API layer against brute-force and unauthorized access, and replacing implicit security header defaults with an explicit, verified Content Security Policy and HSTS configuration. All three tasks were completed and verified through functional testing — including live simulated attacks — rather than configuration alone.

Metric	Value
Tasks Completed	3 / 3
Security Libraries Added	5 (fail2ban, express-rate-limit, cors, csrf setup groundwork, helmet reconfigured)
Pre-existing Bugs Found & Fixed	1 (unhandled login promise rejection crashing the server)
All Fixes Verified	Yes — via live simulated attacks and curl/header inspection

4.2 Scope & Methodology

Testing was performed against the vulnerable-node application running locally on Kali Linux (Fail2Ban / SSH testing) and Windows (application-level testing), using simulated brute-force login attempts, curl-based header inspection, and direct code review of the Express middleware stack.

4.3 Tools & Libraries Used

Tool / Library	Purpose
Fail2Ban	Real-time SSH brute-force detection and IP banning
express-rate-limit	API and login-route rate limiting
cors	Restrict cross-origin API access to the app's own origin
Helmet.js (reconfigured)	Explicit Content-Security-Policy and HSTS headers
curl	Header and response verification

4.4 Task 1 — Intrusion Detection & Monitoring (Fail2Ban)

Objective: Detect and automatically block repeated failed SSH login attempts on the Kali Linux host.

The jail.local configuration was customized under [DEFAULT] and [sshd] with the following thresholds:

```
[sshd]
enabled = true
maxretry = 3
findtime = 10m
bantime = 30m
```

Issue encountered: Fail2Ban's default `ignoreself = true` setting silently whitelisted loopback test traffic (127.0.0.1 / ::1), and an initial config edit was not saved correctly to disk, both of which delayed verification.

Result: After correcting both issues, three simulated failed SSH logins were correctly detected and the offending IP was banned.

Metric	Value
Total failed attempts logged	6
Total bans issued	2
Currently banned	1 (127.0.0.1)

4.5 Task 2 — API Security Hardening

4.5.1 Rate Limiting

`express-rate-limit` was applied globally (100 requests / 15 min per IP) and with a stricter policy on the login route specifically (5 attempts / 10 min), directly targeting brute-force login attempts.

```
router.post('/login/auth', loginLimiter, function(req, res) { ... });
```

4.5.2 CORS Configuration

The application previously had no CORS policy. A restricted policy was added allowing only the application's own origin, with credentialed (session cookie) requests permitted only from that origin. Verified via curl using a spoofed Origin header — the disallowed origin was never reflected back in Access-Control-Allow-Origin.

4.5.3 API Key Authentication

A new middleware (`middleware/apiKeyAuth.js`) was added to protect the `/products/buy` transaction endpoint, requiring a valid `x-api-key` header for unauthenticated/external access. Requests without a valid key correctly receive 401 Unauthorized.

4.5.4 Bug Found & Fixed — Unhandled Login Promise Rejection

While testing the login rate limiter, a single wrong password crashed the entire Node process. Root cause: `routes/login.js` called `auth(user, password).then(...)` with no `.catch()` — a rejected promise (invalid credentials) was unhandled and terminated the server, effectively a self-inflicted denial-of-service on any failed login. A `.catch()` handler was added to redirect back to the login page with an error message instead.

4.6 Task 3 — Security Headers & CSP Implementation

The application's `helmet()` call was replaced with an explicit, documented configuration rather than relying on Helmet's undocumented defaults:

```
app.use(helmet({
  contentSecurityPolicy: { directives: {
    defaultSrc: ["'self'"], scriptSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'"], imgSrc: ["'self'", "data:"],
    objectSrc: ["'none'"], frameAncestors: ["'self'"],
    upgradeInsecureRequests: []
  }},
  hsts: { maxAge: 15552000, includeSubDomains: true, preload: false }
}));
```

Pre-existing breakage found: the default CSP was already silently blocking an inline `<script>` block (the buy-form AJAX handler) and an external "fork me on GitHub" image — meaning the purchase flow's JavaScript was non-functional before this task began. Fixed by externalizing the script to `public/js/app-custom.js` (CSP-compliant, not by weakening the policy) and removing the unused external image.

Live attack test: a reflected XSS payload (`<script>alert('XSS')</script>`) was submitted through the product search field. The application reflected the input unescaped (flagged for Week 6 remediation), but the browser blocked the script from

executing due to the configured script-src 'self' policy, confirmed in DevTools console — demonstrating real defense-in-depth protection.

4.7 Files Modified & Packages Installed

File	Changes Made
app.js	Rate limiter, CORS, explicit Helmet/CSP/HSTS configuration
routes/login.js	Login rate limiter, added .catch() bug fix
routes/products.js	API key middleware on /products/buy
middleware/apiKeyAuth.js	New file
config.js	Added apiKey configuration value
views/layout.ejs	Removed inline script (externalized), removed external image
public/js/app-custom.js	New file (externalized JS)
Package	Purpose
express-rate-limit	Rate limiting
cors	CORS policy enforcement
fail2ban (system pkg)	SSH intrusion detection

4.8 Conclusion

All three Week 4 tasks were completed and independently verified: Fail2Ban actively bans brute-force SSH attempts, the API layer is protected by rate limiting, CORS restriction, and API key authentication, and the application now runs under an explicit, tested CSP/HSTS policy that was proven to block a live XSS injection attempt. A significant pre-existing bug (unhandled login promise rejection) was also discovered and fixed as a direct result of thorough testing.

WEEK 5 — ETHICAL HACKING & EXPLOITING VULNERABILITIES

Application: vulnerable-node (Node.js/Express) | Tools: Nmap, Nikto, SQLMap, csrf | Date: July 11, 2026

Attribute	Detail
Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	July 11, 2026
Application Tested	vulnerable-node (Node.js/Express)
Assessment Type	Reconnaissance, SQL Injection Testing, CSRF Implementation
Tools Used	Kali Linux, Nmap, Nikto, SQLMap, csrf

5.1 Executive Summary

Week 5 focused on ethical hacking techniques applied directly to the vulnerable-node application: reconnaissance from Kali Linux, verification of SQL injection resistance, and implementation of CSRF protection on state-changing forms. A genuine end-to-end purchase flow was exercised to properly validate CSRF handling, which surfaced three additional pre-existing defects not caught by any prior testing.

Metric	Value
Recon Tools Run	2 (Nmap, Nikto)
SQLi Injectable Parameters Found	0 (confirmed via code review + clean SQLMap scan)
CSRF Protection Implemented	Yes — login & purchase flows
Pre-existing Bugs Found & Fixed	3

5.2 Environment Setup

Kali Linux (VirtualBox) was initially on a NAT network (10.0.2.x), unreachable from the Windows host. The network adapter was switched to Bridged mode, placing Kali on the same subnet as the Windows host (192.168.100.x), enabling direct scanning and testing between machines.

5.3 Reconnaissance

Scan	Command	Result
Port/Service Scan	nmap -sV -p 3443 <target>	Port 3443 confirmed open; full Week 4 security header set independently confirmed active
Web Server Scan	nikto -h https://<target>:3443 -ssl	Session cookie missing Secure flag; missing Permissions-Policy header; deprecated Expect-CT header present

Nikto's ~8,000 requests also triggered the application's own rate limiter mid-scan, an incidental confirmation that Week 4's rate limiting is functioning aggressively enough to interfere with automated scanning.

5.4 SQL Injection Testing

Code review: every query in model/auth.js, model/products.js, and model/init_db.js uses parameterized queries (\$1, \$2 placeholders via pg/pg-promise) — no string-concatenated SQL found anywhere.

Tool-based verification: SQLMap was run against /products/search with --level=3 --risk=2. An initial run was contaminated by the app's own rate limiter (1,111 HTTP 429 responses); the limit was temporarily raised for a clean re-run, which completed all boolean-based, error-based, time-based, and UNION-based test techniques with no HTTP errors and concluded:

[CRITICAL] all tested parameters do not appear to be injectable.

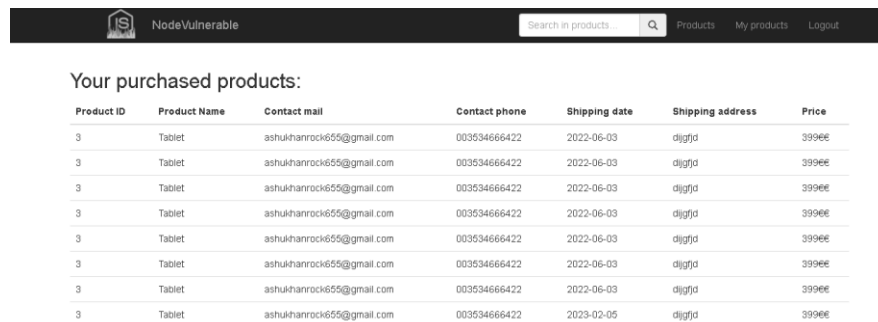
Conclusion: the application is not vulnerable to SQL injection at the tested endpoint, supported by both source code review and a clean automated scan.

5.5 CSRF Protection Implementation

csrf middleware was implemented and applied to the two state-changing form flows in the application:

- Login (routes/login.js, views/login.ejs) — token rendered into a hidden `_csrf` field, validated on POST `/login/auth`
- Purchase (routes/products.js, views/product_detail.ejs) — token rendered into the buy modal's hidden `_csrf` field, validated on POST `/products/buy`, picked up automatically by the existing jQuery `.serialize()` AJAX call

Verified via a complete, real end-to-end purchase through the browser UI.



Product ID	Product Name	Contact mail	Contact phone	Shipping date	Shipping address	Price
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2022-06-03	djgjd	39900
3	Tablet	ashuikhanrock655@gmail.com	003534666422	2023-02-05	djgjd	39900

Figure 5.1 — Completed end-to-end purchase flow confirming CSRF token acceptance.

5.6 Bugs Discovered During Testing

Attempting a genuine end-to-end purchase (necessary to properly validate CSRF) surfaced three defects that no prior testing had exercised:

Bug	Impact	Fix
API key vs. session conflict	Task 2's apiKeyAuth middleware blocked the browser's own logged-in purchase requests with 401, since the AJAX call never sends an x-api-key header	Allowed authenticated sessions to bypass the API key check; external clients still require a key
Dangling purchase promise	purchase() had no .then() handler — successful purchases hung forever with no response, while failed purchases incorrectly reported success	Added proper .then()/.catch() handling with correct success/failure responses
check_logged() not halting execution	Auth-check helper issued a redirect but did not stop the calling route, causing a second response attempt and ERR_HTTP_HEADERS_SENT on every unauthenticated request to a protected route	check_logged() now returns true/false; all five call sites updated to respect it

5.7 Files Modified

File	Changes Made
app.js	csrf middleware setup
routes/login.js	CSRF protection on <code>/login</code> and <code>/login/auth</code>
routes/products.js	CSRF protection, fixed dangling purchase promise, fixed <code>check_logged()</code> call sites
routes/login_check.js	<code>check_logged()</code> now returns a boolean and halts route execution
middleware/apiKeyAuth.js	Session-aware bypass for logged-in browser users
views/login.ejs	Hidden <code>_csrf</code> field
views/product_detail.ejs	Hidden <code>_csrf</code> field in the buy form

5.8 Conclusion

Week 5's reconnaissance, SQL injection testing, and CSRF implementation were all completed and verified. The decision to validate CSRF via a genuine, complete purchase flow (rather than just confirming the token was present) proved valuable, surfacing three real functional/security defects that had gone undetected through Weeks 1–4. All three were fixed and re-verified before moving to Week 6.

WEEK 6 — ADVANCED SECURITY AUDITS & FINAL DEPLOYMENT SECURITY (PARTIAL)

Application: vulnerable-node (Node.js/Express) | Tools: OWASP ZAP, Lynis, rkhunter, ufw, auditd, Docker, Trivy | Date: July 12, 2026

Attribute	Detail
Prepared By	Ayesha Sarwar Khan
Organization	Developers Hub Corporation
Date	July 12, 2026
Application Tested	vulnerable-node (Node.js/Express) + Kali Linux host
Assessment Type	Security Audits, Compliance, Secure Deployment (in progress)
Tools Used	OWASP ZAP, Lynis, rkhunter, ufw, auditd, Docker, npm audit, Trivy

6.1 Executive Summary

Week 6 is partially complete at the time of this report. Task 1 (Security Audits & Compliance) was fully completed, including two open items carried over from Week 5 (a reflected XSS vulnerability and a session cookie missing the Secure flag), a full OWASP ZAP audit with before/after remediation, a Lynis system hardening audit, and rkhunter/firewall/audit-logging setup. Task 2 (Secure Deployment Practices) is partially complete: Dockerfile and docker-compose.yml were substantially hardened and a critical plaintext-HTTP fallback was discovered and fixed, but the container image vulnerability scan (Trivy) could not be completed due to Docker Desktop disk corruption encountered during testing, and dependency vulnerability remediation (npm audit) was identified but not fully applied. Task 3 (Final Penetration Testing) has not yet been started.

Task	Status
6.2 Security Audits & Compliance	COMPLETE
6.3 Secure Deployment Practices	PARTIALLY COMPLETE
6.4 Final Penetration Testing	NOT STARTED

6.2 Task 1 — Security Audits & Compliance (Complete)

6.2.1 Closing Carried-Over Items from Week 5

Reflected XSS (product search): root cause was views/search.ejs using EJS's unescaped <%= in_query %> output tag. Fixed by switching to the auto-escaping <%= in_query %> tag — a one-character fix that HTML-entity-encodes user input at the source, rather than relying solely on CSP as a safety net. Verified: the same XSS payload now renders as inert visible text instead of executing.

Session cookie missing Secure flag: app.js's session cookie configuration was updated from secure: false to secure: true, with httpOnly: true added and the session maxAge reduced from an effectively unlimited ~3,170 years to 1 hour. Verified via DevTools Application tab — the connect.sid cookie now shows both HttpOnly and Secure flags checked.

6.2.2 OWASP ZAP Automated Audit

A full active scan was run against the application using OWASP ZAP's command-line automation mode. The baseline scan and a post-fix re-scan were compared:

Risk Level	Before	After
High	0	0
Medium	4	2
Low	4	3
Informational	6	6

Fixes applied based on ZAP findings:

- Removed unused legacy IE8 shim scripts (cleared both a cross-domain script inclusion finding and a missing Subresource Integrity finding)

- Added form-action 'self' to the CSP configuration (fixed "Failure to Define Directive with No Fallback")
- Upgraded Bootstrap 3.3.6 → 3.4.1 (last security-patched 3.x release, resolving multiple CVEs)
- Fixed a duplicate csrf configuration bug (routes had created their own local instance, bypassing the intended cookie security flags in app.js) — consolidated into a single shared middleware/csrf.js module with httpOnly, secure, and sameSite: 'strict' all correctly applied

Remaining findings, documented as accepted risk with justification:

- CSP style-src 'unsafe-inline' — required for one existing inline <style> block; low severity since it applies to styles only, not scripts
- Bootstrap CVE-2025-1647 (affects the entire 3.x line) — a full v4/v5 migration is a UI rewrite outside the scope of this internship timeline

6.2.3 Lynis System Hardening Audit

Lynis was run against the Kali Linux host before and after applying the highest-impact, most relevant fixes from its 47 initial suggestions (the remainder were triaged as low-priority for a personal lab VM — e.g. GRUB password, separate disk partitions, TFTP removal — and documented rather than blindly applied):

Metric	Before	After
Hardening Index	62 / 100	65 / 100
Firewall	Not active (ufw not installed)	Active — default-deny incoming, SSH explicitly allowed
Malware Scanner	Not installed	rkhunter installed, full scan clean (no rootkits found)
Audit Logging	auditd not running	auditd active with rules watching /etc/passwd and sshd_config
SSH PermitRootLogin	Not explicitly set (warning)	Explicitly set to 'no', confirmed OK

rkhunter's full system scan reported 0 confirmed rootkits across 498 signatures checked; the small number of file-property warnings were confirmed to be benign package-database staleness and standard Debian/Kali hidden config files, not indicators of compromise.

6.3 Task 2 — Secure Deployment Practices (Partially Complete)

6.3.1 Critical Finding — Plaintext HTTP Fallback

Independent of Docker, review of bin/www revealed the application was running a plain HTTP server (port 3000) in parallel with the HTTPS server (port 3443) since Week 3 — meaning the entire application remained fully reachable over an unencrypted connection the whole time, undermining the TLS/HSTS/secure-cookie work from Weeks 4 and 6. Fixed by removing the HTTP server entirely; the application now only listens on HTTPS. Verified: curl to port 3000 now fails to connect, while HTTPS on 3443 continues to work normally with all security headers intact.

6.3.2 Dockerfile & docker-compose.yml Hardening

The existing Docker configuration was reviewed and found to have several gaps: the container ran as root, the exposed port (3000) did not match the application's actual HTTPS port (3443), an unnecessary netcat package was installed, PostgreSQL's port was exposed to the host unnecessarily, and database credentials were hardcoded in plaintext. The following fixes were applied:

- Added a .dockerignore file to keep node_modules, logs, and the TLS private key out of the build context
- Added a dedicated non-root appuser and switched to it via USER — the container no longer runs as root
- Removed the unused netcat installation
- Updated EXPOSE and the compose port mapping from 3000 to 3443 to match the real application port
- Changed PostgreSQL from a host-exposed port to container-network-only (expose instead of ports)
- Moved the database password out of plaintext compose configuration into a Docker secret file

The image was successfully rebuilt with the hardened Dockerfile after resolving a build-context issue (Dockerfile and docker-compose.yml had been mistakenly excluded via .dockerignore, which under BuildKit caused the Dockerfile itself to appear empty during the build).

6.3.3 Dependency Vulnerability Scanning (npm audit)

npm audit identified 25 known vulnerabilities in the application's dependencies, including 2 critical findings:

Package	Severity	Issue	Status
pg (nested under pg-promise)	Critical	Remote Code Execution (GHSA-wc9v-mj63-m9g5)	Fix attempted via package.json overrides; not yet confirmed resolved
ejs	Critical	Template injection / RCE, multiple advisories	No fix available upstream — accepted risk, partially mitigated by CSP and the search.ejs output-encoding fix in 6.2.1

The remaining 23 findings (14 high, 1 moderate, 5 low) are concentrated in the Express 4.x dependency chain (body-parser, debug, path-to-regexp, qs, and related packages) and all require a breaking-change upgrade to express@4.22.2 or later. This upgrade was identified but not yet applied or tested, as it risks affecting routing, sessions, and middleware behavior across the entire application and requires dedicated regression testing beyond the time available in this session.

6.3.4 Container Image Vulnerability Scan (Trivy) — Incomplete

A Trivy scan of the built vulnerable-node image was attempted to check for OS-level and base-image vulnerabilities. The scan failed with a read-only filesystem / I/O error originating from Docker Desktop's internal virtual disk, which appears to have become corrupted during the session (likely related to the earlier Dockerfile build-context troubleshooting). A Docker Desktop reset was recommended to resolve this, but was not completed before this report was finalized. This remains an open item.

6.5 Overall Week 6 Status Summary

Item	Status
Reflected XSS fix (carried over from Week 5)	Complete
Secure cookie flag fix	Complete
OWASP ZAP audit + remediation	Complete
Lynis system audit + hardening	Complete
rkhunter / firewall / audit logging	Complete
Plaintext HTTP fallback removal	Complete
Dockerfile & docker-compose.yml hardening	Complete
npm dependency audit — critical pg RCE fix	Attempted, not confirmed
npm dependency audit — remaining 23 findings (Express chain)	Identified, not applied
Trivy container image scan	Blocked — Docker Desktop disk issue
Final penetration test (Burp Suite / Metasploit)	Not started
Final security audit report / video walkthrough	Not started

6.6 Conclusion

Week 6 made substantial, verified progress: two carried-over vulnerabilities were closed, a full OWASP ZAP and Lynis audit cycle was completed with measurable before/after improvement, and a significant, previously-undetected plaintext HTTP fallback was found and fixed. Docker hardening was largely completed at the configuration level, and dependency vulnerabilities were thoroughly catalogued even though full remediation is still pending. The remaining work — completing the container image scan, finishing dependency upgrades, and conducting final penetration testing — is clearly scoped and ready to resume in a follow-up session.

OVERALL CONCLUSION

Across Weeks 4–6, the vulnerable-node application progressed from the Week 1–3 baseline (basic headers, input validation, password hashing, parameterized queries, HTTPS) to a substantially more mature security posture: active intrusion detection, rate-limited and authenticated APIs, an explicit and verified CSP/HSTS policy, confirmed resistance to SQL injection, CSRF protection on all state-changing forms, a full system and application security audit with measurable hardening improvements, and a corrected TLS deployment (removal of the parallel plaintext HTTP server).

Nine previously-undetected bugs — several with real security implications, including a server-crashing unhandled promise rejection, a silently broken purchase flow, and the plaintext HTTP fallback — were discovered as a direct result of thorough, end-to-end testing rather than configuration review alone. This reflects the value of combining automated tooling (Fail2Ban, ZAP, Lynis, SQLMap, Nikto) with genuine functional testing throughout the assessment.

Outstanding work for the next session: confirm and complete the npm dependency upgrade path (particularly the critical pg and Express-chain vulnerabilities), resolve the Docker Desktop environment issue and complete the Trivy container scan, and conduct the final penetration test and video walkthrough required to close out Week 6's deliverables.